

Document number:	P0621R0
Date:	2017-03-02
Project:	C++ Extensions for Ranges, Library Working Group
Reply-to:	Casey Carter < Casey@Carter.net >

Ready Ranges TS Issues

258: Remove subsection “C library algorithms” from <experimental/ranges/algorithm>

The C library is not in scope for the TS, this material was inadvertently incorporated by editorial error.

Proposed Resolution

Strike section [alg.c.library]. Remove the reference thereto in Table 62 and in the descriptive text in [algorithms.general]/2.

244 move_iterator::operator* should be defined in terms of iter_move

This change is required for correct behavior of move_iterator<I> when I is a proxy iterator, it was overlooked by P0022.

Proposed Resolution

Change the synopsis of class move_iterator in [move.iterator]:

```
template <InputIterator I>
class move_iterator {
public:
    [...]
    using iterator_category = input_iterator_tag;
    using reference = rvalue_reference_t<I> see below;
    move_iterator();
    [...]
};
```

Delete paragraph 1:

1 Let R be reference_t<I>. If is_reference<R>::value is true, the template specialization move_iterator<I> shall define the nested type named reference as a synonym for remove_reference_t<R>&&, otherwise as a synonym for R;

Change the description of move_iterator::operator* in [move.iter.op.star] as follows:

1 *Effects*: Equivalent to: `return iter_move(current); static_cast<reference>(*current);`

Change the description of move_iterator::operator[] in [move.iter.op.star] as follows:

1 *Effects*: Equivalent to: `return iter_move(i + n); static_cast<reference>(current[n]);`

243 difference_type of arrays

The is_array specialization for difference_type is extraneous; the fallback/deducing specialization handles array types properly.

Proposed Resolution

Change [iterator.traits]/4 as follows:

```
[...]

template <class T>
struct difference_type<T*>
    : enable_if<is_object<T>::value, ptrdiff_t> { };

template <class I>
requires is_array<I>::value
struct difference_type<I> : difference_type<decay_t<I>> { };

template <class I>
```

```
struct difference_type<I const> : difference_type<decay_t<I>> { };
```

[...]

242 Customizing `iter_move` and `iter_swap` is needlessly complicated

The specifications of `iter_move` and `iter_swap` require user customizations found via ADL to compete in overload resolution with the “default” implementations which are constrained functions that accept forwarding references. This makes it challenging for users who are not expert in the rules for partial ordering of constrained function templates to hook the customization points.

Proposed Resolution

Replace the text of [iterator.utils.iter_move] with the following:

1. The name `iter_move` denotes a *customization point object* ([customization.point.object]). The effect of the expression `ranges::iter_move(E)` for some expression `E` is equivalent to the following:
 2. `iter_move(E)`, if that expression is well-formed when evaluated in a context that does not include `ranges::iter_move` but does include the lookup set produced by argument-dependent lookup ([basic.lookup.argdep]).
 3. Otherwise, if `E` is dereferenceable
 - (3.1) If `*E` is an lvalue, `std::move(*E)`
 - (3.2) Otherwise, `*E`
 4. Otherwise, `ranges::iter_move(E)` is ill-formed.
5. If `ranges::iter_move(E)` does not equal `*E`, the program is ill-formed with no diagnostic required.

Replace the text of [iterator.utils.iter_swap] with the following:

1. The name `iter_swap` denotes a *customization point object* ([customization.point.object]). The effect of the expression `ranges::iter_swap(E1, E2)` for some expressions `E1` and `E2` is equivalent to the following:
 2. `(void)iter_swap(E1, E2)`, if that expression is well-formed when evaluated in a context that does not include `ranges::iter_swap` but does include the lookup set produced by argument-dependent lookup ([basic.lookup.argdep]) and the following declaration:

```
void iter_swap(auto, auto) = delete;
```
 3. Otherwise, if the expressions `E1` and `E2` both satisfy `Readable`, and if the reference type of `E1` is swappable with ([concepts.lib.corelang.swappable]) the reference type of `E2`, then `ranges::swap(*E1, *E2)`
 4. Otherwise, if the types `T1` and `T2` of `E1` and `E2` satisfy `IndirectlyMovableStorable<T1, T2>()` && `IndirectlyMovableStorable<T2, T1>()`, exchanges the values denoted by `E1` and `E2`.
 - (4.1) `ranges::iter_swap(E1, E2)` is a constant expression if and only if `E1` and `E2` are constant expressions and the required operations of `IndirectlyMovableStorable<T1, T2>()` and `IndirectlyMovableStorable<T2, T1>()` are constant expressions.
 - (4.2) `ranges::iter_swap(E1, E2)` is `noexcept` if and only if `E1` and `E2` are `noexcept` and the required operations of `IndirectlyMovableStorable<T1, T2>()` and `IndirectlyMovableStorable<T2, T1>()` are `noexcept`.
 5. Otherwise, `ranges::iter_swap(E1, E2)` is ill-formed.
6. If `ranges::iter_swap(E1, E2)` does not swap the values denoted by the expressions `E1` and `E2`, the program is ill-formed with no diagnostic required.

241 IndirectlySwappable is broken

IndirectlySwappable as defined in N4620:

```
template <class I1, class I2 = I1>
concept bool IndirectlySwappable() {
    return Readable<I1>() && Readable<I2>() &&
        requires (const I1 i1, const I2 i2) {
            ranges::iter_swap(i1, i2);
            ranges::iter_swap(i2, i1);
        }
};
```

```

        ranges::iter_swap(i1, i1);
        ranges::iter_swap(i2, i2);
    };
}

```

Requires that it is possible to `iter_swap` *constant* `I1` and `I2`. This indirectly requires `Readable<const I1>` and `Writable<const I1>` (resp. `I2`) both of which require `Movable`. Since constant objects cannot be the target of assignment operations, `Movable<const I1>` (resp. `I2`) is not satisfied. This issue only manifests when `I1` and `I2` are non-reference types, creating a confusing discrepancy between `IndirectlySwappable<Foo, Bar>` and `IndirectlySwappable<Foo&&, Bar>`.

This problem does not manifest in the implementations because neither applies `const` to the parameter types before passing them to `iter_swap`.

`IndirectlySwappable<int[2], int[4]>()` is also incorrectly satisfied due to array-to-pointer decay of function (and hence requires expression) parameters. This should be corrected by accepting the `requires` clause parameters by forwarding reference.

Proposed Resolution

Change the definition of `IndirectlySwappable` in `[commonalgoreq.indirectlyswappable]` as follows:

```

template <class I1, class I2 = I1>
concept bool IndirectlySwappable() {
    return Readable<I1>() && Readable<I2>() &&
        requires (const I1 i1, const I2 i2) {
            ranges::iter_swap(i1, i2);
            ranges::iter_swap(i2, i1);
            ranges::iter_swap(i1, i1);
            ranges::iter_swap(i2, i2);
        }
        requires (I1&& i1, I2&& i2) {
            ranges::iter_swap(std::forward<I1>(i1), std::forward<I2>(i2));
            ranges::iter_swap(std::forward<I2>(i2), std::forward<I1>(i1));
            ranges::iter_swap(std::forward<I1>(i1), std::forward<I1>(i1));
            ranges::iter_swap(std::forward<I2>(i2), std::forward<I2>(i2));
        }
};
}

```

240 Should Writable require Semiregular, or “Move-Defaultable”

P0022R2 changed the `Readable` concept by replacing the `Semiregular` requirement with `Movable` and `DefaultConstructible`; this allows move-only types like `unique_ptr<int>` and `optional<thread>` to model `Readable`.

P0022R2 notably did *not* make the symmetric change to the `Writable` concept. Why should it be forbidden to write to these move-only types from which I may read?

Note that this discrepancy is exacerbated by the fact that both implementations ([cmcstl2](#), [range-v3](#)) of the TS have *already* implemented the same relaxation for `Writable`.

Proposed Resolution

Modify the definition of `Writable` (`[iterators.writable]`) as follows:

```

template <class Out, class T>
concept bool Writable() {
    return Semiregular<Out>() &&
        return Movable<Out>() && DefaultConstructible<Out>() &&
        requires (Out o, T&& t) {
            *o = std::forward<T>(t); // not required to be equality preserving
        };
};
}

```

239 Remove the “Experimental additional constraints” from Readable

From [#183](#):

We can probably just remove these sanity checks from `range-v3`, `cmcstl2`, and the Ranges TS. I added them to keep myself honest when I was designing and implementing proxy iterator support. Now they only serve to make compile times worse. Holler if you see a reason to keep them.

I see no reason to keep them; we should strike those requirements from P0022 before moving it into the TS.

Proposed Resolution:

Wording relative to P0022R2. Change the definition of the Readable concept in [iterators.readable] as follows:

```
template <class I>
concept bool Readable() {
    return Movable<I>() && DefaultConstructible<I>() &&
        requires (const I& i) {
            typename value_type_t<I>;
            typename reference_t<I>;
            typename rvalue_reference_t<I>;
            { *i } -> Same<reference_t<I>>;
            { iter_move(i) } -> Same<rvalue_reference_t<I>>;
        } &&
    // Relationships between associated types
    CommonReference<reference_t<I>, value_type_t<I>&>() &&
    CommonReference<reference_t<I>, rvalue_reference_t<I>>() &&
    CommonReference<rvalue_reference_t<I>, const value_type_t<I>&>() &&
    Same<
        common_reference_t<reference_t<I>, value_type_t<I>>,
        value_type_t<I>>() &&
    Same<
        common_reference_t<rvalue_reference_t<I>, value_type_t<I>>,
        value_type_t<I>>();
}
```

238 P0022 broke indirect_result_of

By removing my undocumented fix to indirect_result_of that remove_references the function type before passing it to IndirectCallable:

```
template <class F, class... Is>
    requires IndirectCallable<remove_reference_t<F>, Is...>()
struct indirect_result_of<F(Is...)> :
    result_of<F(value_type_t<Is>...)> { };
```

Without that remove_reference_t, when F is a reference type, we require the trivially satisfiable CopyConstructible<F> instead of the desired CopyConstructible<remove_reference_t<F>>.

indirect_result_of is used directly in the declarations of the transform algorithms, and in projected. As a result, after the change, transform doesn't properly require the transformation function - and no algorithm properly requires its projection arguments - to be CopyConstructible.

Proposed Resolution:

In section [indirectcallables.indirectfunc], change the definition of indirect_result_of to read:

```
template <class F, class... Is>
    requires IndirectCallable<decay_t<F>, Is...>()
struct indirect_result_of<F(Is...)> :
    result_of<F(reference_t<Is>...)> { };
```

237 IndirectCallable should use value_type_t<I>& instead of value_type_t<I>

In the Proxy Iterators paper, IndirectCallable is defined as:

```
template <class F, class I>
concept bool IndirectCallable() {
    return Readable<I>() &&
        Callable<F, value_type_t<I>>() &&
        Callable<F, reference_t<I>>() &&
        Callable<F, iter_common_reference_t<I>>();
}
```

I strongly suspect this should be:

```
template <class F, class I>
concept bool IndirectCallable() {
    return Readable<I>() &&
        Callable<F, value_type_t<I>&>() &&
```

```

    Callable<F, reference_t<I>>>() &&
    Callable<F, iter_common_reference_t<I>>>();
}

```

N.B. `value_type_t<I>&`. In the rest of the section as well.

Reason: algorithms generally do not call predicates with rvalues of the value type. More often, they create a (non-const) local to cache a value, and then pass *that* to a predicate. Hence, `IndirectCallable` should be testing with `value_type_t<I>&`.

Proposed Resolution

Change every instance of `value_type_t<XXX>` in `[indirectcallables.indirectfunc]` to `value_type_t<XXX>&` (see #189 for detailed wording.)

236 projected<I>::value_type incorrectly decays arrays to pointers

The following code does not compile with `projected` as it is currently defined in the latest draft.

```

int matrix[3][4] = {};
std::experimental::ranges::for_each(matrix, [](int(&)[4]){});

```

Proposed Resolution

Modify the definition of `projected` in `[projected]` as follows:

```

template <Readable I, IndirectRegularCallable<I> Proj>
struct projected {
    using value_type = decay_t<remove_cv_t<remove_reference_t<indirect_result_of_t<Proj&(I)>>>>;
    indirect_result_of_t<Proj&(I)> operator*() const;
};

```

189 concept callable should perfectly forward its function to invoke

Originally, the function & callable concepts treated functions as lvalues since the algorithms accept functions by value, and therefore *always* invoke lvalue expressions. Given that we will almost certainly be using the `Callable` concepts with non-algorithm library components, and we want to be somewhat consistent with `is_callable`, I think we have enough reason to change that behavior to perfectly forward. (See the github issue tracker for complete discussion.)

Proposed Resolution

(The proposed resolution is relative to P0022R2 (proxy iterators) and also includes the resolution to #237.)

Change the definition of `Callable` in `[concepts.lib.callables.regularcallable]` as follows:

```

template <class F, class... Args>
concept bool Callable() {
    return CopyConstructible<F>() &&
        requires(F&& f, Args&&... args) {
            invoke(std::forward<F>(f), std::forward<Args>(args)...); // not required to be equality preserving
        };
}

```

Change the definition of `Predicate` in `[concepts.lib.callables.predicate]` as follows:

```

template <class F, class... Args>
concept bool Predicate() {
    return RegularCallable<F, Args...>() &&
        Boolean<result_of_t<F&(Args...)>>();
}

```

Change the definition of `IndirectCallable` in `[indirectcallables.indirectfunc]` to:

```

template <class F>
concept bool IndirectCallable() {
    return CopyConstructible<F>() && Callable<F&>();
}
template <class F, class I>
concept bool IndirectCallable() {
    return Readable<I>() && CopyConstructible<F>() &&

```

```

    Callable<F, value_type_t<I>>() &&
    Callable<F, reference_t<I>>() &&
    Callable<F, iter_common_reference_t<I>>());
}
template <class F, class I1, class I2>
concept bool IndirectCallable() {
    return Readable<I1>() && Readable<I2>() &&
    CopyConstructible<F>() &&
    Callable<F, value_type_t<I1>, value_type_t<I2>>() &&
    Callable<F, value_type_t<I1>, reference_t<I2>>() &&
    Callable<F, reference_t<I1>, value_type_t<I2>>() &&
    Callable<F, reference_t<I1>, reference_t<I2>>() &&
    Callable<F, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

```

Change IndirectRegularCallable in likewise fashion.

Change IndirectCallablePredicate to:

```

template <class F, class I>
concept bool IndirectCallablePredicate() {
    return Readable<I>() &&
    CopyConstructible<F>() &&
    Predicate<F, value_type_t<I>>() &&
    Predicate<F, reference_t<I>>() &&
    Predicate<F, iter_common_reference_t<I>>();
}
template <class F, class I1, class I2>
concept bool IndirectCallablePredicate() {
    return Readable<I1>() && Readable<I2>() &&
    CopyConstructible<F>() &&
    Predicate<F, value_type_t<I1>, value_type_t<I2>>() &&
    Predicate<F, value_type_t<I1>, reference_t<I2>>() &&
    Predicate<F, reference_t<I1>, value_type_t<I2>>() &&
    Predicate<F, reference_t<I1>, reference_t<I2>>() &&
    Predicate<F, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

```

Change IndirectCallableRelation to:

```

template <class F, class I1, class I2 = I1>
concept bool IndirectCallableRelation() {
    return Readable<I1>() && Readable<I2>() &&
    CopyConstructible<F>() &&
    Relation<F, value_type_t<I1>, value_type_t<I2>>() &&
    Relation<F, value_type_t<I1>, reference_t<I2>>() &&
    Relation<F, reference_t<I1>, value_type_t<I2>>() &&
    Relation<F, reference_t<I1>, reference_t<I2>>() &&
    Relation<F, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

```

Change IndirectCallableStrictWeakOrder to:

```

template <class F, class I1, class I2 = I1>
concept bool IndirectCallableStrictWeakOrder() {
    return Readable<I1>() && Readable<I2>() &&
    CopyConstructible<F>() &&
    StrictWeakOrder<F, value_type_t<I1>, value_type_t<I2>>() &&
    StrictWeakOrder<F, value_type_t<I1>, reference_t<I2>>() &&
    StrictWeakOrder<F, reference_t<I1>, value_type_t<I2>>() &&
    StrictWeakOrder<F, reference_t<I1>, reference_t<I2>>() &&
    StrictWeakOrder<F, iter_common_reference_t<I1>, iter_common_reference_t<I2>>();
}

```

Change the signatures of the generate and generate_n algorithms as follows:

```

template <CallableClass F, OutputIterator<result_of_t<F&()>>Iterator 0, Sentinel<0> S>
requires Callable<F&() && Writable<0, result_of_t<F&()>>()
0 generate(0 first, S last, F gen);

template <CallableClass F, OutputRange<result_of_t<F&()>>class Rng>
requires Callable<F&() && OutputRange<Rng, result_of_t<F&()>>()
safe_iterator_t<Rng>
generate(Rng&& rng, F gen);

template <CallableClass F, OutputIterator<result_of_t<F&()>>Iterator 0>
requires Callable<F&() && Writable<0, result_of_t<F&()>>()
0 generate_n(0 first, difference_type_t<0> n, F gen);

```